the presented value will be needed by choosing a sensible magnitude factor. For example, consider the following two answers calculated for the size of a file:

a 34 560 bytes

Here, a conversion to kibibytes would be sensible using the calculation:

$$344560\text{B} = \frac{34560}{1024}\text{KiB} = 33.75\,\text{KiB}$$

b 3 456 000 bytes

Here, a conversion to mebibytes would be sensible using the calculation:

$$3456000\text{B} = \frac{\left(\frac{3456000}{1024}\right)}{1024}\text{MiB} = 3.296\,\text{MiB}$$

If a calculation is to be performed with values quoted with different magnitude factors there must first be conversions to ensure all values have the same magnitude factor. For example, if you needed to know how many files of size 2.4 MiB could be stored on a 4 GiB memory stick there should be a conversion of the GiB value to the corresponding MiB value.

The calculation would be:

$$\frac{(4 \times 1024)\text{MiB}}{2.4\text{MiB}} = 1076$$

1.03 Internal coding of numbers

The discussion in this chapter relates only to the coding of integer values. The coding of non-integer numeric values (real numbers) is considered in Chapter 16 (Section 16.03).

Coding for integers

Computers need to store integer values for a number of purposes. Sometimes only a simple integer is stored, with the understanding that it is a positive number. This is stored simply as a binary number. The only decision to be made is how many bytes should be used. If the choice is to use two bytes (16 bits) then the range of values that can be represented is 0 to $(2^{16} - 1)$ which is 0 to 65 535.

However, in many cases we need to identify whether the number is positive or negative, so we use a signed integer. A signed integer can just have the binary code for the value with an extra bit to define the sign. This is referred to as 'sign and magnitude representation'. For this the convention is to use a 0 to represent + and a 1 to represent –. A few examples of this are shown in Table 1.08.

However, there are a number of disadvantages to using this format, so signed integers are usually in **two's complement** form. Here we need two definitions.

The **one's complement** of a binary number is defined as the binary number obtained if each binary digit is individually subtracted from 1. This means that each 0 is switched to 1 and each 1 switched to 0. The two's complement is defined as the binary number obtained if 1 is added to the one's complement number.

KEY TERMS

One's complement: the binary number obtained by subtracting each digit in a binary number from 1

Two's complement: the one's complement of a binary number, plus 1

If you need to convert a binary number to its two's complement form, you can use the method indicated by the definition but there is a quicker method. For this you start at the least significant bit and move left ignoring any zeros up to the first 1, which you also ignore. Change any remaining bits from 0 to 1 or from 1 to 0.

For example, expressing the number 10100100 in two's complement form leaves the right-hand 100 unchanged, then the remaining 10100 changes to 01011, so the result is 01011100.

To represent a positive denary integer value as the equivalent two's complement binary form, the process is as follows.

- Use one of methods from Section 1.01 to convert the denary value to a binary value.
- Add a 0 in front of this binary value.

To represent a negative denary integer value as the equivalent two's complement binary form the process is as follows.

- Disregard the sign and use one of methods from Section 1.01 to convert the denary value to a binary value.
- Add a 0 in front of this binary value.
- Convert this binary value to its two's complement form.

A few simple examples of two's complement representations are shown in Table 1.08.

To convert a two's complement binary number representing a positive value into a denary value, the leading zero is ignored and one of the methods in Section 1.01 is applied to convert the remaining binary.

There are two alternative methods for converting a two's complement binary number representing a negative number into a denary value. These are illustrated in Worked Example 1.03.

WORKED EXAMPLE 1.03

Methods for converting a negative number expressed in two's complement form to the corresponding denary number

Consider the two's complement binary number 10110001.

Method 1. Convert to the corresponding positive binary number then convert to denary before adding the minus sign

- Converting 10110001 to two's complement leaves unchanged the 1 in the least significant bit position then changes all of the remaining bits to produce 01001111.
- You ignore the leading zero and apply one of the methods from Section 1.01 to convert the remaining binary to denary which gives 79.
- You add the minus sign to give −79.

Method 2. Sum the individual place values but treat the most significant bit as a negative value

You follow the approach illustrated in Table 1.02 to convert the original binary number 10110001 as follows:

Place value	-2^7 = −128	2^6 = 64	2^5 = 32	2^4 = 16	2^3 = 8	2^2 = 4	2^1 = 2	2^0 = 1
Digit	1	0	1	1	0	0	0	1
Product	−128	0	32	16	0	0	0	1

You now add the values in the bottom row to get −79.

Some points to note about two's complement representation are as follows.

- There is only one representation of zero.
- Starting from the lowest negative value, each successive higher value is obtained by adding 1 to the binary code. In particular, when all digits are 1 the next step is to roll over to an all-zero code. This is the same as any digital display would do when each digit has reached its maximum value.
- Just adding a leading zero to an unsigned binary value converts it to the two's complement representation of the corresponding positive number
- You use a two's complement conversion to change the sign of a number from positive to negative or from negative to positive. We say that the two's complement values are self-complementary.
- You can add any number of leading zeros to a representation of a positive value without changing the value.
- You can add any number of leading ones to a representation of a negative value without changing the value.

Signed denary number to be represented	Sign and magnitude representation	Two's complement representation
7	0111	0111
1	0001	0001
0	0000	0000
–0	1000	Not represented
–1	1001	1111
–7	1111	1001
–8	Not represented	1000

Table 1.08 Representations of signed integers



TIP

If you are converting a negative denary number into two's complement you begin by converting the denary value to a binary value. Then you must not forget to add a leading zero before taking the two's complement to convert the positive value to a negative value.

TASK 1.02

Take the two's complement of the binary code for –7 and show that you get the code for +7.

TASK 1.03

Convert the two's complement number 1011 to the denary equivalent. Then do the same for 111011 and convince yourself that you get the same value.

Discussion Point:

What is the two's complement of the binary value 1000? Are you surprised by this?

Binary arithmetic

Before considering the addition of binary numbers it is useful to recall how we add two denary numbers. Two rules apply. The first rule is that the process is carried out starting with addition of the two least significant digits and then working right to left. The second rule is that if an addition produces a value greater than 9 there is a carry of 1. For example in the addition of 48 to 54, the first step is adding 8 to 4 to get 2 with a carry of 1. Then 5 is added to 4 plus the carried 1 to give 0 with carry 1. The rules produce 102 for the sum which is the correct answer.

For binary addition, starting at the least significant position still applies. The rules for the addition of binary digits are:

- 0 + 0 = 0
- 0 + 1 = 1
- 1 + 1 = 0 with a carry of 1
- 1 + 1 + 0 = 0 with a carry of 1
- 1 + 1 + 1 = 1 with a carry of 1

The last two rules are used when a carried 1 is included in the addition of two digits.

As an example, the addition of the binary equivalent of denary 14 to the binary equivalent of denary 11 can be examined.

```
      1  0  1  1
+     1  1  1  0
  ---------------
  1   1  0  0  1
```

The steps followed from right to left are:

- 1 + 0 = 1 with no carry
- 1 + 1 = 0 with carry 1
- 0 + 1 + carried 1 = 0 with carry 1
- 1 + 1 + carried 1 = 1 with carry 1

The rules have correctly produced the 5-bit answer which is the binary equivalent of 25. In a paper exercise like this these rules for addition will always produce the correct answer.

Again for subtraction we can first consider how this is done for denary numbers. As for addition the process starts with the least significant digits and proceeds right to left. The special feature of subtraction is the "borrowing" of a 1 from the next position when a subtracting digit is larger than the digit it is being subtracted from.

For example in subtracting 48 from 64 the first step is to note that 8 is larger than 4. Therefore 1 has to be borrowed as 10. The 10 added to 4 gives 14 and 8 subtracted from this gives 6. When we proceed to the next digit subtraction we first have to reduce the 6 to 5 because of the borrow. So we have subtraction of 4 from 5 leaving 1. The answer for the subtraction is 16.

For binary subtraction, starting at the least significant position still applies. The rules for the subtraction of binary digits are:

- 0 – 0 = 0
- 0 – 1 = 1 after a borrow
- 1 – 0 = 1
- 1 – 1 = 0

As an example, the subtraction of the binary equivalent of denary 11 from the binary equivalent of denary 14 can be examined.

	1	1	1	0
−	1	0	1	1
	0	0	1	1

The steps followed from right to left are:

- 1 is larger than 0 so 1 is borrowed giving subtraction of 1 from 10 leaving 1
- Because of the borrow the 1 is reduced to 0 so that 1 is to be subtracted from 0. This requires a further borrow giving subtraction of 1 from 10 leaving 1
- Because of the borrow the 1 is reduced to 0 leaving subtraction of 0 from 0
- 1 – 1 gives 0

The answer is the binary value for denary 3.

When binary addition is carried out by a computer using internally stored numbers there is a major difference. This arises from the fact that the storage unit will always have a defined number of bits. For example, in the above addition, if binary values were limited to being stored in a nibble the result of the addition would be incorrectly stored as 1001. This is an example of an **overflow**. The value produced is too large to be stored.



KEY TERM

Overflow: a condition when the result of a calculation is too large to fit into the number of bits defined for storage

When the values in a computer system are stored in two's complement form this problem has a characteristic behaviour.

In the following addition where +63 is added to +63 there is no problem; the answer is correctly obtained as +126:

	0	0	1	1	1	1	1	1
+	0	0	1	1	1	1	1	1
	0	1	1	1	1	1	1	0

However, if the binary for +96 is added to +96 the result is as follows:

	0	1	1	0	0	0	0	0
+	0	1	1	0	0	0	0	0
	1	1	0	0	0	0	0	0

The overflow means that the answer has a leading 1, which causes a computer system to interpret the answer as a negative number.

A similar problem can occur when two negative values are added. For example the addition of −96 to the same value results in the following:

		1	0	1	0	0	0	0	0
+		1	0	1	0	0	0	0	0
	(1)	0	1	0	0	0	0	0	0

This time there has been a carry when the most significant bits were added and the result obtained is a positive number.

Clearly we need the processor to detect overflow and output an error message. There is a discussion of how a processor can detect overflow in Chapter 6 (Section 6.07).

One of the advantages of using two's complement representations is that it simplifies the process of subtracting one number from another. The number being subtracted is converted to its two's complement form, which is then added to the other number.

TASK 1.04

Using a byte to represent each value, carry out the subtraction of denary 35 from denary 67 using binary arithmetic with two's complement representations.

Binary coded decimal (BCD)

One exception to grouping bits in bytes to represent integers is the **binary coded decimal (BCD)** scheme. This is useful in applications that require single denary digits to be stored or transmitted. The BCD code uses a nibble to represent a denary digit. We consider the simple scheme where the digits are coded as the binary values from 0000 to 1001. The remaining codes 1010 to 1111 do not have any meaning.

KEY TERMS

Binary coded decimal (BCD): storage of a binary value representing one denary digit in a nibble

Packed BCD: when two BCD nibbles are stored in one byte

If a denary number with more than one digit is to be converted to BCD there has to be a group of four bits for each denary digit. There are, however, two options for BCD; the first is to store one BCD code in one byte, leaving four bits unused. The other option is **packed BCD** where two 4-bit codes are stored in one byte. Thus, for example, the denary digits 8503 could be represented by either of the codes shown in Figure 1.01.

One BCD digit per byte	00001000	00000101	00000000	00000011
Two BCD digits per byte	10000101	00000011		

Figure 1.01 Alternative BCD representations of the denary digits 8503

There are a number of applications where BCD can be used. The obvious type of application is where denary digits are to be displayed, for instance on the screen of a calculator or in a digital time display. A somewhat unexpected application is for the representation of currency values. When a currency value is written in a format such as $300.25 it is as a fixed-point decimal number (ignoring the dollar sign). It might be expected that such values would be stored as real numbers but this cannot be done accurately (this type of problem is discussed in more detail in Chapter 16 (Section 16.03)). One solution to the problem is to store each denary digit as a BCD code.

Let's consider how BCD arithmetic might be performed by a computer if fixed-point decimal values for currency were stored as BCD values. Here is an example of addition.

0.26	0000 0000	0010 0110	
		+	
0.85	0000 0000	1000 0101	
		↓	
	0000 0000	1010 1011	

Figure 1.02 Incorrect addition using BCD coding

We will assume a two-byte packed BCD representation. The first byte represents two denary digits for the whole part of the number and the second byte represents two denary digits for the fractional part. If the two values are $0.26 and $0.85 then the result of the addition should be $1.11. This would involve a carry from the first decimal place to the whole number 1. However, applying simple binary addition of the BCD codes would produce the result shown in Figure 1.02.

The additions for the fractional parts have produced values corresponding to the denary values 10 and 11 but a BCD value is supposed to be a single digit. The error has resulted in no carry to the whole number column.

We need the processor to recognise that an impossible value has been produced and apply a method to correct this. The solution is to add 0110 whenever the problem is detected. This is illustrated in Figure 1.03.

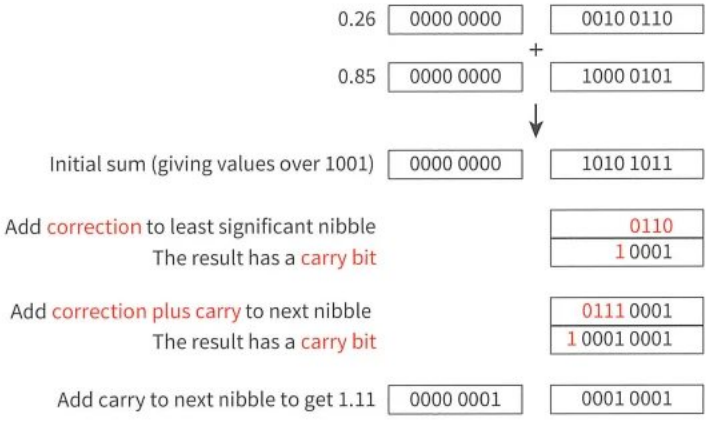


Figure 1.03 Use of the correction value to perform BCD addition

The steps shown in Figure 1.03 are as follows.

- Starting with the least significant nibble, adding 0110 to 0101 gives 1011 which is recognised as being incorrect.
- The 0110 correction value is added to produce 10001.
- The 0001 is stored and the leading 1 is carried to the next nibble.
- In the first decimal position adding 0100 to 1000 then adding the carry bit 1 gives 1011 which is recognised as being incorrect.
- The 0110 correction is added to produce 10001.
- The 0001 is stored and the leading 1 is carried to the next nibble.

In this example the two whole number nibbles have zero values so adding these has no effect.

1.04 Internal coding of text

To store text in a computer, we need a coding scheme that provides a unique binary code for each distinct individual component item of the text. Such a code is referred to as a character code. There have been many different examples of character coding schemes throughout the history of computing.